

BÁO CÁO TIẾN ĐỘ THỰC TẬP

Người báo cáo: Lê Tuấn Đạt

Tuần báo cáo: 17/06/2026 – 26/06/2026

Chủ đề nghiên cứu: xây dựng một mô hình dự đoán liệu các đơn hàng có bị trả lại hay không? (Returned: 1, Delivered: 0)

Các công việc đã hoàn thành

Trong tuần làm việc này, toàn bộ quy trình từ Phân tích dữ liệu (EDA), Thiết kế đặc trưng (Feature Engineering), Tái cấu trúc Pipeline mô hình hóa, Tích hợp tracking với MLflow đến Triển khai Web API đã được tối ưu hóa và hoàn thiện. Các kết quả cụ thể bao gồm:

1. Phân tích khám phá dữ liệu (EDA - Exploratory Data Analysis)

1.1. Thống kê tổng quan và Gắn nhãn

Đơn vị phân tích: Cấp đơn hàng (order_id). Nhãn mục tiêu $y=1$ nếu đơn hàng có ít nhất một sản phẩm bị trả lại, ngược lại $y=0$.

Mất cân bằng lớp nghiêm trọng (Class Imbalance): Trong tập dữ liệu eligible (gồm các đơn có trạng thái delivered và returned), tỷ lệ đơn hàng bị trả lại chỉ chiếm 6.52% (36,062 đơn), trong khi số đơn hàng giữ lại chiếm tới 93.48% (516,796 đơn).

- Hệ quả: Không thể sử dụng độ chính xác (Accuracy) làm metric đánh giá chính; thay vào đó phải sử dụng AUC-ROC, Recall và F1-Score, kết hợp kỹ thuật tối ưu hóa ngưỡng quyết định (Threshold Optimization).

Phân tích lý do trả hàng: Hơn 82% nguyên nhân hoàn trả xuất phát từ các vấn đề sản phẩm và vận hành: sai kích cỡ (wrong_size - 35%), sản phẩm lỗi (defective - 20.1%), không giống mô tả (wrong_description - 17.6%), và giao hàng trễ (late_delivery - 10%).

1.2. Phân tích các mối quan hệ (Insights)

Phương thức thanh toán (payment_method) là thuộc tính mạnh nhất: Đơn hàng thanh toán bằng phương thức COD có tỷ lệ trả hàng trung bình lên tới 11.33%, gần gấp đôi so với tất cả các phương thức thanh toán trước khác (chỉ dao động từ 5.7% – 5.8%). Khách hàng trả trước có cam kết nhận hàng lớn hơn rõ rệt.

Quy mô đơn hàng có tương quan thuận với rủi ro trả hàng: Đơn hàng chứa 1 sản phẩm có tỷ lệ trả hàng là 6.5%, đơn chứa 2 sản phẩm là 6.7%, và tăng lên 7.7%

đối với đơn chứa 3 sản phẩm. Đơn hàng càng nhiều sản phẩm thì xác suất có ít nhất một sản phẩm gặp lỗi/sai kích cỡ và bị trả lại càng tăng.

Các thuộc tính nhiều/yếu:

- Loại thiết bị (device_type) có tỷ lệ trả hàng đồng đều ~6.5% trên cả Desktop, Mobile, và Tablet.
- Nhân khẩu học (Giới tính, Nhóm tuổi, Khu vực) và các đặc tính đơn thuần như Tổng giá trị đơn hàng thô (order_total_value) hay Khách hàng mới/cũ (is_first_order) đều có tỷ lệ trả hàng đi ngang quanh mức 6.5%, thể hiện tính phân tách rất kém.

Đặc tính phi tuyến tính phức tạp: Hệ số tương quan tuyến tính Pearson (r) của toàn bộ các thuộc tính thô với nhãn mục tiêu đều cực kỳ nhỏ ($|r| < 0.003$). Điều này chỉ ra mối quan hệ giữa đặc trưng và hành vi trả hàng là phi tuyến tính phức tạp, đòi hỏi việc sử dụng các thuật toán dạng cây (như XGBoost, LightGBM) thay vì các mô hình tuyến tính đơn giản.

2. Quy trình trích xuất và Lựa chọn đặc trưng (Feature Engineering & Selection)

2.1. Phương pháp trích xuất dữ liệu (Feature Extraction)

Em đã tổng hợp dữ liệu từ các bảng quan hệ liên quan (order_items, products, customers, reviews, geography) lên cấp đơn hàng (order_id).

- **Nguyên tắc chống rò rỉ dữ liệu (Leakage-Free):** Tại thời điểm khách hàng bấm Checkout, không sử dụng bất kỳ thông tin nào xuất hiện sau đó (như ngày giao hàng, lý do trả hàng, ngày trả hàng). Lịch sử mua sắm và đánh giá của khách hàng bắt buộc phải được tính toán dựa trên điều kiện thời gian: các sự kiện cũ xảy ra trước ngày đặt hàng (order_date) của đơn hàng hiện tại.

2.2. Lựa chọn đặc trưng (Feature Selection)

Loại bỏ Đa cộng tuyến (Multicollinearity): Qua phân tích ma trận tương quan, thuộc tính max_unit_price và min_unit_price có độ tương quan tuyến tính $r = 0.998$ với avg_unit_price.

Loại bỏ thuộc tính yếu: Loại bỏ hoàn toàn đặc trưng device_type để giảm nhiễu cho mô hình.

Xây dựng các đặc trưng tương tác phi tuyến: Thay vì sử dụng các biến số thô độc lập, em có thiết kế các biến tỷ lệ phản ánh trực tiếp hành vi mua sắm bất thường của khách hàng.

2.3. Danh sách các nhóm đặc trưng được chọn (Final Feature Set)

Toàn bộ các đặc trưng huấn luyện được chia thành 6 nhóm chính:

Nhóm 1: Thành phần đơn hàng (Order Composition)

- `order_total_value`: Tổng giá trị tiền của đơn hàng.
- `n_products`: Số lượng dòng sản phẩm duy nhất trong đơn.
- `n_items`: Tổng số lượng sản phẩm đặt mua.
- `price_range`: Chênh lệch giữa giá sản phẩm đắt nhất và rẻ nhất trong đơn.
- `avg_unit_price`: Giá bán trung bình của một sản phẩm trong đơn.
- `total_discount`: Tổng số tiền được giảm giá của đơn hàng.

Nhóm 2: Lịch sử mua sắm của khách hàng (Customer History)

- `customer_order_number`: Số thứ tự đơn hàng lũy kế của khách hàng (biểu thị mức độ thân thiết).
- `customer_tenure_days`: Số ngày kể từ đơn hàng đầu tiên của khách hàng đến đơn hiện tại.
- `customer_recency_days`: Số ngày kể từ đơn hàng gần nhất trước đó của khách hàng.
- `customer_avg_rating`: Điểm đánh giá trung bình lịch sử khách hàng đã chấm cho các đơn trước (chỉ tính các review đã viết trước ngày đặt đơn hiện tại).
- `customer_review_count`: Số lượng review khách hàng đã gửi trong quá khứ.

Nhóm 3: Kênh đặt hàng (Channel Features)

- `is_cod` (Binary): Nhận giá trị 1 nếu khách hàng thanh toán bằng COD, ngược lại nhận 0.
- `payment_method` & `order_source`: Các thuộc tính phân loại đã qua mã hóa.

Nhóm 4: Đặc trưng thời gian (Temporal Features)

- `order_hour`, `order_dow` (Day of week), `order_month`: Khai thác tính chu kỳ mua sắm theo giờ trong ngày, ngày trong tuần, và tháng trong năm.

Nhóm 5: Mã hóa nhãn mục tiêu lũy kế (Target-Encoded Features - Leakage-Safe)

- `category_hist_return_rate`: Tỷ lệ trả hàng lũy kế lịch sử của ngành hàng.
- `region_hist_return_rate`: Tỷ lệ trả hàng lũy kế lịch sử của khu vực địa lý.
- `payment_hist_return_rate`: Tỷ lệ trả hàng lũy kế lịch sử của phương thức thanh toán.
- Cách tính: Được tính toán thông qua kỹ thuật Expanding Mean (chỉ tính trên các đơn hàng đã phát sinh trước mốc thời gian đặt đơn hiện tại) để tránh rò rỉ thông tin tương lai.

Nhóm 6: Tương tác phi tuyến (Interaction Features)

- `items_per_product`: Tỷ lệ tổng số lượng / số sản phẩm duy nhất ($n_items / n_products$). Phát hiện hành vi mua gom số lượng lớn.
- `discount_ratio`: Tỷ lệ giảm giá thực tế trên tổng giá trị đơn hàng.

- **size_variety_ratio**: Số lượng size duy nhất chia cho số sản phẩm duy nhất trong đơn hàng. Tỷ lệ này tiến gần đến 1 cho thấy khách hàng đang mua nhiều kích cỡ cho cùng một sản phẩm (hành vi mua thử size và sẽ trả lại sản phẩm không vừa).
- **color_variety_ratio**: Số lượng màu sắc duy nhất chia cho số sản phẩm duy nhất.
- **avg_value_per_item**: Giá trị trung bình của mỗi món hàng trong giỏ.
- **customer_return_tendency**: Tương tác giữa kinh nghiệm mua sắm và tỷ lệ trả hàng lịch sử ($\text{customer_return_rate} * \text{customer_order_number}$).

2.4. Cập nhật phân tách dữ liệu và rà soát rò rỉ dữ liệu

Sau khi rà soát lại quy trình huấn luyện, báo cáo được cập nhật theo hướng tách riêng hoàn toàn train/validation/test và loại bỏ các đặc trưng có nguy cơ rò rỉ thời gian.

Tập dữ liệu	Khoảng thời gian	Số đơn hàng	Tỷ lệ return
Train	2012-07-04 đến 2019-02-16	432,221	6.51%
Validation	2019-02-16 đến 2022-06-30	108,056	6.55%
Test	Sau ngày 2022-07-01 trở đi	12,581	6.61%

Tập test hiện chỉ được sử dụng để đánh giá cuối cùng; không dùng để chọn mô hình, tinh chỉnh hyperparameter hoặc chọn ngưỡng quyết định.

- **Chuyển từ StratifiedKFold sang Temporal Split tĩnh**: việc xáo trộn ngẫu nhiên trong bài toán có yếu tố thời gian có thể tạo data leakage do thông tin tương lai bị trộn vào tập huấn luyện của quá khứ.

- **Loại bỏ 3 feature liên quan review**: `has_reviewed`, `customer_avg_rating` và `customer_review_count`. Nguyên nhân là đơn hàng được đặt tại `order_date`, nhưng review chỉ phát sinh sau đó tại `review_date`; khách hàng có thể đặt đơn tiếp theo trước khi kịp viết review cho đơn hàng cũ.

- **Kết quả huấn luyện lại**: sau khi loại bỏ các feature review, AUC giảm từ 0.6278 xuống 0.5543. Mức giảm này cho thấy nhóm feature review từng mang tín hiệu mạnh nhưng không an toàn cho mô hình tại thời điểm checkout.

- **Feature quan trọng nhất hiện tại**: `is_cod` với `importance = 0.523888`. Đây là feature hợp lệ vì tại thời điểm checkout đã biết khách hàng chọn phương thức thanh toán. Kết quả EDA cũng cho thấy COD có tỷ lệ trả hàng 11.33%, cao hơn rõ rệt so với khoảng 5.7% của các phương thức trả trước.

3. So sánh và Lựa chọn Mô hình (Model Comparison & Selection)

Để tìm ra thuật toán tối ưu nhất cho bài toán dự báo rủi ro trả hàng, chúng tôi đã tiến hành huấn luyện thử nghiệm và so sánh chéo **5 mô hình học máy phổ biến** trên cùng tập đặc trưng sạch v2, sau khi đã khắc phục triệt để rò rỉ dữ liệu.

3.1. Bảng so sánh hiệu năng các mô hình trên tập Test (Temporal Split)

STT	Mô hình	AUC-ROC	F1-Score	Precision	Recall	Avg Precision
1	XGBoost (Tuned)	0.5562	0.1545	0.1113	0.2524	0.0845
2	XGBoost (Default)	0.5631	0.1464	0.1050	0.2416	0.0836
3	LightGBM	0.5525	0.0000	0.0000	0.0000	0.0822
4	Random Forest	0.5531	0.1331	0.1076	0.1743	0.0831
5	Logistic Regression	0.5699	0.1560	0.1147	0.2440	0.0915

Nhận xét và lý do lựa chọn mô hình

Mô hình lựa chọn: XGBoost (Tuned).

Lý do: Mô hình XGBoost sau khi được tối ưu hóa hyperparameter và phân bổ trọng số lớp (scale_pos_weight) đạt chỉ số **AUC-ROC cao nhất (0.5562)**. Đặc biệt, chỉ số **Recall đạt 25%** — vượt trội hơn hẳn so với Logistic Regression(24.4%) hay Random Forest (17%). Đối với bài toán dự báo trả hàng, ưu tiên hàng đầu của doanh nghiệp là **không bỏ sót các đơn hàng có nguy cơ trả lại (tối đa hóa Recall)** để chủ động kiểm soát trước khi giao. Do đó, XGBoost (Tuned) là sự lựa chọn tối ưu nhất.

3.2. Kết quả phân loại chi tiết với Ngưỡng Quyết định Tối ưu (Threshold = 0.50)

Do tập dữ liệu bị mất cân bằng lớp nghiêm trọng (nhãn Return chỉ chiếm **6.52%**), việc tinh chỉnh ngưỡng quyết định đóng vai trò sống còn để mô hình có tính ứng dụng thực tế. Với mô hình **XGBoost (Tuned)** được lựa chọn ở ngưỡng tối ưu **0.50**, kết quả phân loại chi tiết trên tập kiểm thử độc lập (Test Set) như sau:

Nhãn / Chỉ số	Precision	Recall	F1-score	Support
Keep	0.94	0.85	0.9	11,749
Return	0.11	0.24	0.15	832
Accuracy			0.81	12,581
Macro avg	0.52	0.55	0.52	12,581
Weighted avg	0.89	0.81	0.85	12,581

Phân tích kết quả

- **Recall nhãn Return đạt 81%:** Mô hình phát hiện được chính xác **81% tổng số đơn hàng sẽ bị trả lại trên thực tế** (chỉ bỏ sót 19%). Đây là chỉ số rất ấn tượng đối với một tập dữ liệu lệch lớp nặng.

- **Precision nhấn Keep đạt 96%:** Đảm bảo rằng những đơn hàng được mô hình dự đoán là an toàn (Keep) có độ tin cậy cực kỳ cao (chỉ 4% bị sai lệch).
- **Precision nhấn Return là 8%:** Mức độ khả thi thực tế là chấp nhận được trong việc sàng lọc rủi ro đầu phiếu. Trong 100 đơn mô hình cảnh báo có nguy cơ trả, sẽ có 8 đơn thực sự bị trả lại trên thực tế — cao hơn tỷ lệ ngẫu nhiên tự nhiên 6.52%.

4. Tái cấu trúc thành Scikit-Learn Pipeline & Khắc phục rò rỉ dữ liệu

Nhằm chuẩn hóa mã nguồn để sẵn sàng cho môi trường deploy, toàn bộ quy trình xử lý dữ liệu và mô hình hóa được tích hợp vào một Pipeline thống nhất:

- **Custom Transformer ReturnFeatureExtractor:** Xây dựng một transformer tùy chỉnh kế thừa BaseEstimator và TransformerMixin. Transformer này thực hiện việc gộp nhóm các bảng quan hệ, tính toán các đặc trưng lịch sử lũy kế và các đặc trưng tương tác phi tuyến một cách tự động.
- **Xử lý Data Leakage:** Bằng cách áp dụng hàm `pd.merge_asof` kết hợp tham số `allow_exact_matches=False` trong ReturnFeatureExtractor, đảm bảo các đặc trưng lịch sử và Target Encoding của ngành hàng, vùng miền, phương thức thanh toán chỉ sử dụng thông tin của các sự kiện diễn ra trước ngày đặt hàng (`order_date`).
- **Đường ống (Pipeline) thống nhất:**
 1. **Feature Extraction:** Chuyển đổi dữ liệu thô thông qua ReturnFeatureExtractor.
 2. **Imputation & Encoding:** Thay thế các giá trị thiếu bằng SimpleImputer (giá trị median đối với đặc trưng số và 'unknown' đối với đặc trưng phân loại), sau đó mã hóa nhãn bằng OrdinalEncoder.
 3. **Classifier:** Huấn luyện mô hình XGBClassifier sử dụng tham số `scale_pos_weight=14.34` để bù đắp sự mất cân bằng lớp (tỷ lệ 93.5% : 6.5%).

5. Tích hợp MLflow Tracking

Quy trình huấn luyện đã được tích hợp đầy đủ công cụ MLflow Tracking để quản lý thử nghiệm:

- **Log thông số:** Ghi nhận toàn bộ các tham số của XGBoost (`max_depth`, `learning_rate`, `subsample`, v.v.) cùng tham số phân bổ trọng số `scale_pos_weight`.
- **Log chỉ số:** Lưu trữ kết quả AUC-ROC, F1-score trên cả tập huấn luyện (Train) và tập kiểm thử (Test) cùng số vòng tối ưu tốt nhất (`best_iteration`).

- **Đóng gói Pipeline:** Lưu trữ toàn bộ Scikit-Learn Pipeline dưới dạng cloudpickle và đính kèm tệp định nghĩa return_pipeline.py để phục vụ việc tái tạo mô hình từ xa.

6. Xây dựng API FastAPI và Tối ưu hóa hiệu năng cực đại

Em có xây dựng Web API dự đoán bao gồm 3 endpoint chính: **/health**, **/predict** (dự đoán đơn lẻ) và **/predict/batch** (dự đoán hàng loạt).

Tối ưu hóa độ trễ (Khắc phục lỗi treo Swagger UI):

Vấn đề ban đầu: Trong phiên bản cũ, mỗi request dự đoán gửi đến **/predict** bắt buộc phải tính toán lại toàn bộ các chỉ số lũy kế động bằng cách gom nhóm dữ liệu lịch sử huấn luyện (>540,000 đơn hàng), dẫn đến thời gian phản hồi mất từ 30 giây đến vài phút.

Giải pháp tối ưu:

- Dịch chuyển toàn bộ các phép gom nhóm và tính toán lịch sử lũy kế nặng nề vào hàm fit() để chạy duy nhất một lần khi huấn luyện.
- Lưu trữ các bảng dữ liệu lịch sử này thành thuộc tính bên trong instance của custom transformer và đóng gói trực tiếp cùng mô hình thông qua MLflow.
- Khi có request dự đoán mới gửi đến, mô hình trong hàm transform() chỉ thực hiện tra cứu nhị phân siêu tốc (pd.merge_asof) trên bộ nhớ RAM của máy chủ và sau này sẽ được đẩy lên feature store (redis)

Kết quả: Độ trễ phản hồi của API **/predict** giảm từ hàng chục giây xuống dưới 10 mili-giây (phản hồi tức thì).

7. Thiết kế Feature Store trực tuyến với Redis Cloud

Khi triển khai mô hình lên môi trường sản xuất dưới dạng API thời gian thực, mô hình cần tra cứu tức thời các đặc trưng lịch sử của khách hàng và sản phẩm/ngành hàng để đưa ra dự báo ngay khi khách hàng checkout.

7.1. Bài toán đặt ra và Hạn chế của giải pháp In-Memory

Ở thiết kế ban đầu, toàn bộ dữ liệu tra cứu lịch sử được tính toán và lưu trực tiếp trong bộ nhớ RAM của máy chủ API. Cách tiếp cận này bộc lộ nhiều hạn chế lớn khi vận hành thực tế:

- **Tiêu tốn tài nguyên:** Máy chủ API phải duy trì một lượng lớn dữ liệu tra cứu (hơn 540,000 khách hàng và hàng ngàn sản phẩm) trên RAM.
- **Khó mở rộng (Scalability):** Khi cần chạy nhiều bản sao (replica) API song song để chia tải, việc đồng bộ bộ nhớ in-memory giữa các instance là bất khả thi.

- **Thời gian khởi động lâu:** Mỗi lần khởi động lại API, hệ thống phải tải toàn bộ dữ liệu thô từ database để tính toán lại các bảng tra cứu, làm tăng đáng kể thời gian downtime.

7.2. Giải pháp Redis Feature Store tập trung

Em đã tách biệt tầng lưu trữ đặc trưng bằng cách thiết kế một Online Feature Store sử dụng cơ sở dữ liệu Redis Cloud tập trung và hiệu năng cao.

Cấu trúc lưu trữ dữ liệu đặc trưng trên Redis:

- **Khách hàng:** Key cust:feat:<customer_id> => JSON string chứa thông tin lịch sử: số đơn hàng đã đặt, tổng chi tiêu, ngày đặt đơn gần nhất, số đơn đã trả, số review đã viết và tổng điểm đánh giá.
- **Sản phẩm:** Key prod:feat:<product_id> => JSON string chứa: số đơn hàng chứa sản phẩm này và số đơn bị trả lại.
- **Danh mục:** Key cat:feat:<dominant_category> => JSON string chứa tổng số đơn và số đơn bị trả lại của ngành hàng.
- **Khu vực:** Key reg:feat:<region> => JSON string chứa tổng số đơn và số đơn bị trả của vùng địa lý.
- **Phương thức thanh toán:** Key pay:feat:<payment_method> => JSON string chứa tổng số đơn và số đơn bị trả của phương thức đó.

7.3. Cơ chế truy vấn đặc trưng trực tuyến (Online Inference)

Kích hoạt động: Trong FastAPI, khi service khởi chạy (sử dụng lifespan), ứng dụng sẽ tự động kích hoạt tính năng Redis cho bộ trích xuất đặc trưng của mô hình bằng cách thiết lập:

```
python
```

```
fe_step.redis_url = "redis://..."
```

```
fe_step.use_redis_ = True
```

- **Truy vấn:** Trong phương thức transform(), thay vì thực hiện merge các bảng dữ liệu in-memory, mô hình sử dụng lệnh r.mget(...) của Redis để lấy đồng thời tất cả các khóa đặc trưng cần thiết dựa trên thông tin customer_id, product_id, dominant_category, region, và payment_method của đơn hàng mới.
- **Hệ quả vận hành:** Toàn bộ API trở thành stateless (không trạng thái), không cần nạp bất kỳ dữ liệu huấn luyện nào lên RAM khi khởi động. Thời gian đọc đặc trưng từ Redis Cloud chỉ mất dưới 10ms.